Introduction solutions

Each identifier corresponds to the separate exercise file. Conditions and counterexamples are part of the answer.

OS01-E01 Enforcing a shared-machine boundary

1. Bob must not permanently monopolize the CPU/change privilege, arbitrarily read/write Alice's RAM, or arbitrarily access her files. These related boundaries have separate enforcement points.
2. Protected timer/mode and the kernel scheduler enforce CPU control. MMU/PTE permissions and OS-managed mappings enforce RAM access. Protected entry, trusted credentials, and ACLs/capabilities enforce file operations. Hardware does not directly interpret pathnames; it protects the checking kernel and metadata.
3. A globally shared clipboard or file improves convenience but adds authority and disclosure routes. Grant explicit rights to specified objects instead. Hiding GUI controls cannot enforce an API.

The answer must identify an enforcement point for each resource.

OS01-E02 Events, modes, and context

1. Timer interrupts are external/asynchronous; a system call is synchronous with the current instruction. DMA completion and CPU discovery are distinct events. Polling or interrupts can notify the CPU.
2. A timer interrupt in kernel mode requires neither user→kernel transition nor process switch. A cache-hit read normally has user→kernel→user, two mode transitions, and can return to the same thread with zero process switches. Additional scheduling is counted separately.
3. Polling completion status works if cache/memory ordering and buffer lifetime are correct. Poll frequency, event rate, detection latency, and CPU cost determine the trade-off. DMA does not require one interrupt per block.

Event classification does not substitute for counting actual transitions.

OS01-E03 Hash counterexamples and lifetime

1. Bucket1 is[5,9,13]. Finding13 takes three comparisons; rejecting17 also takes three. n congruent keys give O(n) worst case. Average O(1) needs hash/load assumptions.
2. A maintained balanced ordered tree gives O(log n) worst-case search and range queries. Whether it is preferable for exact-key lookup also depends on costs and synchronization.
3. Unlinking removes future list reachability but does not destroy acquired references. Immediate free/reuse permits use-after-free. Reclaim under a suitable lock, reference-count protocol, or reader-compatible grace period. Stale values are different from invalid memory.

Query cost and lifetime are independent obligations.

OS01-E04 Overlapping two jobs

1. A uses CPU 0–2, I/O2–8, CPU 8–10; B runs10–14. Total14, CPU utilization8/14=4/7.
2. A0–2, B2–6, idle6–8, A8–10; I/O remains2–8. Total10, utilization8/10=0.8. The saved4 units are B's overlap with I/O.
3. No ready B, shared-I/O queueing, switching/synchronization overhead, or memory pressure changes the premises. One CPU still performs8 total CPU units without parallel CPU bursts.

List idle intervals explicitly.

OS01-E05 Cache and NUMA denominators

1. Additional-penalty model gives1+0.1×50=6 cycles. Total-miss-cost model gives0.9×1+0.1×50=5.9 cycles. These formulas use different contracts.
2. 0.8×100+0.2×180=116 ns;0.2×100+0.8×180=164 ns.
3. No. Whole-program time also depends on memory stalls' baseline-time fraction, overlap, caches, bandwidth, and all other work. One subsystem's cost does not automatically equal total execution time.

Keep units and premises.

OS01-E06 Equal VA, unequal data

1. VPN=1, offset0x234=564. P gives0x7000+0x234=0x7234; Q gives0x9234.
2. P's permission allows the write. Q takes a permission fault. Do not allocate a new page merely to bypass prohibited writing. COW requires separate policy markers and conditions absent here.
3. No. A remaining mapping can otherwise access a frame reused for someone else's data. Old references, TLB entries, and I/O must no longer use the frame before it enters the free pool.

Residency, permission, and sharing are distinct.

OS01-E07 Timer and fairness premises

1. With fixed rotation, no higher-priority insertion, and terminating service, a just-missed job waits at most(n−1)q+n c. Changing n or unbounded nonpreemptible execution invalidates the bound.
2. Each q of work incurs c switching. The fraction is0.1/4.1=1/41≈2.44%. It is not a measurement of all context switches.
3. No. A nonterminating handler prevents return to a progressing scheduler. Interrupt serviceability, bounded kernel blocking, and a selection policy without permanent starvation are also needed.

Hardware control recovery does not replace scheduler policy.

OS01-E08 Lost update and BST range

1. T1load0, T2load0, T1store1, T2store1 produces1. A single CPU can switch between steps. Suitable atomic RMW or mutex changes the update to an indivisible specification.
2. Although12>parent5, it belongs to root 10's left subtree while12>10. Search follows right20 then its empty left child, missing existing12. The invariant covers whole subtree ranges.
3. One bit per block gives2^20 bits/8=2^17 B=128 KiB. This is logical bitmap size, not one bit of physical I/O per operation.

Local comparisons do not establish a general invariant.

OS01-E09 Deadlines and platform boundaries

1. A0–1, B1–3, idle3–4, A4–5, idle5–6, B6–8, A8–9, idle9–12. A finishes1,5,9 before4,8,12. B finishes3,8 before6,12. LCM12 and no unfinished boundary work permit repetition.
2. No. U=1/4+2/6=7/12 states demand only. An arbitrary unfair scheduler can idle or choose wrongly. Feasibility, algorithmic guarantees, and actual premises need separate proofs.
3. VMs normally have guest kernels and a VMM boundary. Conventional containers share a host kernel and isolate views through namespaces/resource controls. They do not each necessarily have an independent OS kernel.

Timing correctness is stronger than a low mean.

OS01-E10 API, version, and source availability

1. No. The OSI definition also requires specified redistribution and derived-work rights. Source availability alone is insufficient; inspect each component's license.
2. No. Fix version, architecture, and configuration; consult allowed API outcomes and implementation, then compare general invariants and condition-specific observations. One success excludes neither legal error branches nor races/failures.
3. Success may reach only volatile cache. Define file, metadata, device-cache flush ordering, compliant device behavior, necessary namespace persistence, and the failure model. A successful return alone is insufficient.

Evidence categories cannot replace each other.

OS01-E11 Print jobs and two completion times

The printer serves A from 0 to 6. B joins the queue at t=1, starts at t=6 and finishes at t=8, so its queue wait is 6−1=5 seconds. Submission confirms only the specified acceptance milestone, not printing. No persistence or recovery guarantee is given, so survival after power loss cannot be inferred. A spool manages pending jobs and order; a buffer temporarily holds data to reconcile rates or batch transfers. They can coexist. Different ordering, rates, or job interleaving require a new calculation.

OS01-E12 Concurrency, parallelism, and shared state

In A, P and Q make interleaved progress over 0–4, hence are concurrent; at most one CPU job runs on the sole core at a time, so those CPU jobs are not parallel. In B both execute simultaneously over 0–2 on separate cores: concurrent and parallel. Completion times are 4 and 2 seconds, while total CPU work stays 2+2=4 core-seconds. A non-atomic read-modify-write can lose an update; use synchronization appropriate to the shared contract. Parallelism alone does not make the update atomic.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 1. Open Source Initiative: Open Source Definition.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study